



Universidade Federal de Mato Grosso
Campus Universitário de Várzea Grande

Algoritmos e Estrutura de Dados I

Aula 2 - Introdução à linguagem C

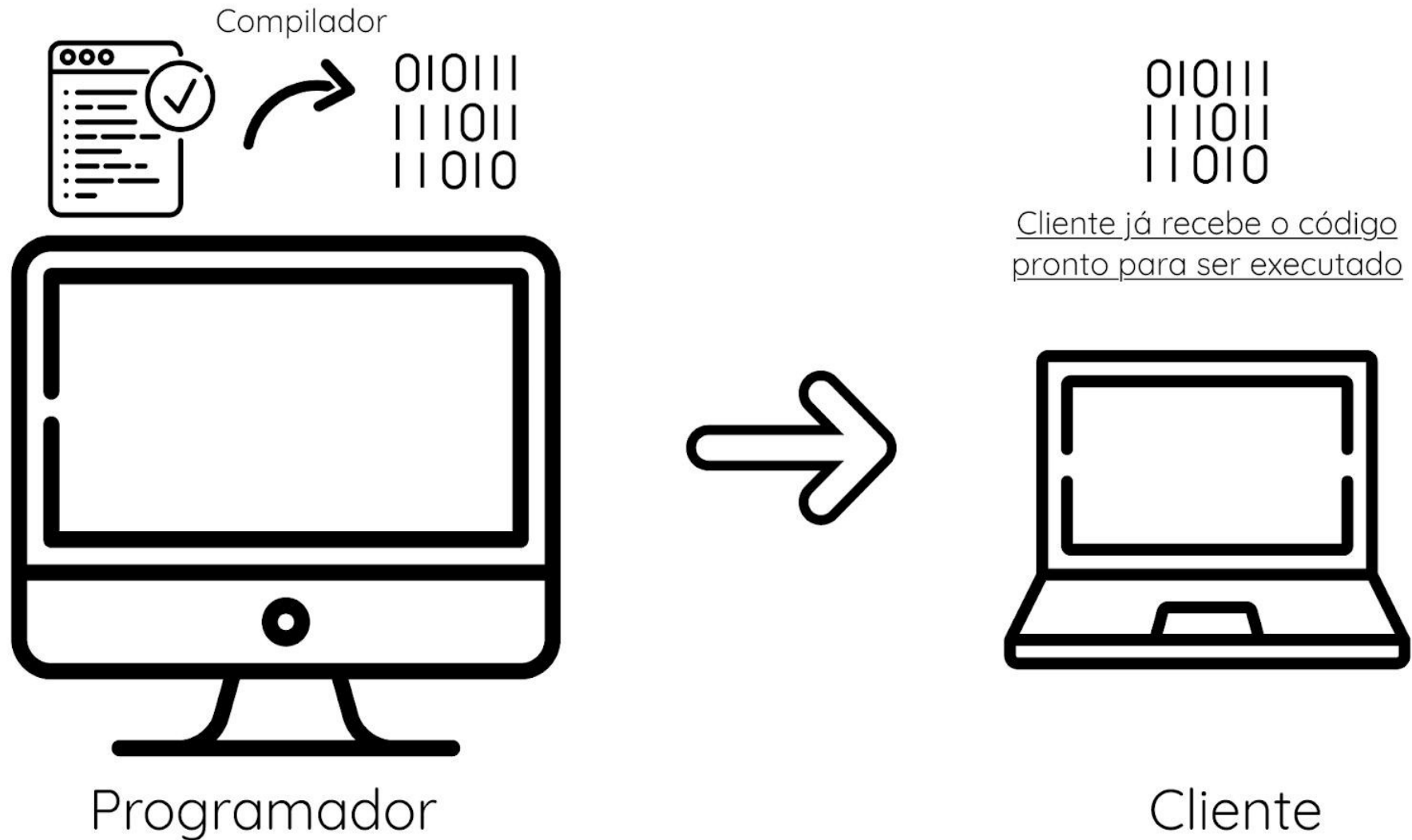
1. Definição de programação
2. Características da linguagem C
3. Tipos de dados e variáveis

O que é programação?

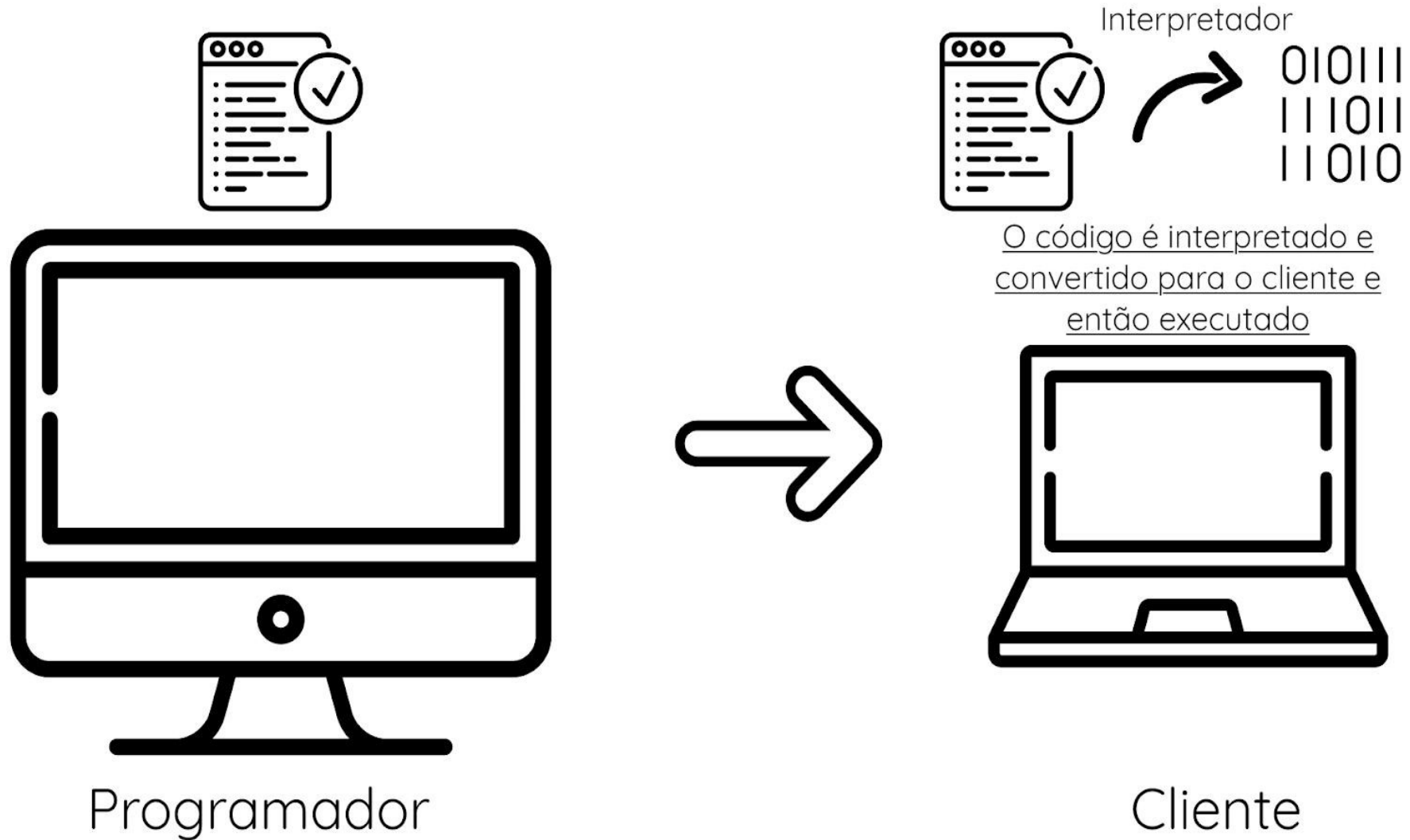
“É o processo de criação de um conjunto de instruções que dizem ao computador como realizar uma tarefa”



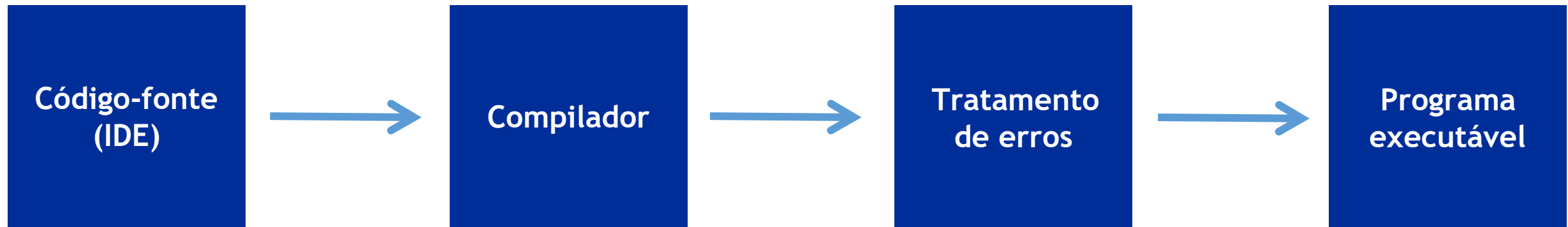
Linguagem compilada vs Linguagem interpretada



Linguagem compilada vs Linguagem interpretada



Passo a passo de um programa



Execução de um programa em .c

```
#include <stdio.h>

int main() {

printf("Hello world");

return 0;
}
```

Arquivo .c

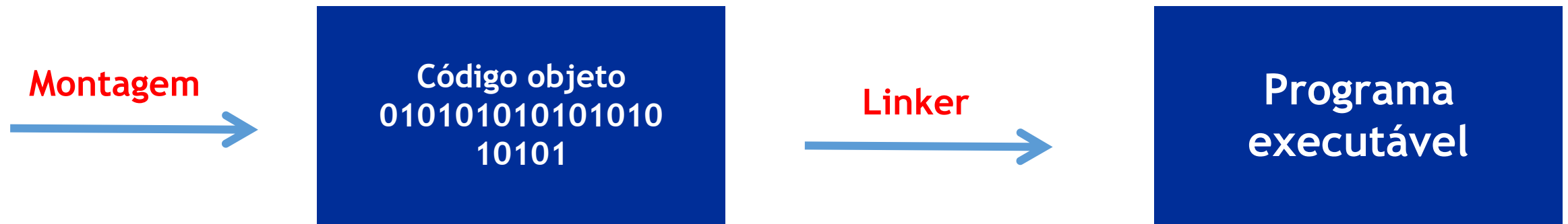
Compilação



```
| .file "teste.c"
| .text
| .def __main; .scl 2; .type 32; .endef
| .section .rdata,"dr"
|
| .LC0:
| .ascii "Hello world\0"
| .text
| .globl main
| .def main; .scl 2; .type 32; .endef
| .seh_proc main
|
| main:
| pushq %rbp
| .seh_pushreg %rbp
| movq %rsp, %rbp
| .seh_setframe %rbp, 0
| subq $32, %rsp
| .seh_stackalloc 32
| .seh_endprologue
| call __main
| leaq .LC0(%rip), %rcx
| call printf
| movl $0, %eax
| addq $32, %rsp
| popq %rbp
| ret
| .seh_endproc
| .ident "GCC: (x86_64-posix-seh-rev0, Built by MinGW-W64 project) 8.1.0"
| .def printf; .scl 2; .type 32; .endef
```

Arquivo .s

Execução de um programa em .c



Estrutura básica de um arquivo .c

```
#include <stdio.h>
```

```
int main () {
```

```
|
```

```
return 0;
```

```
}
```

Regras gerais

- ❑ A linguagem C é case-sensitive
- ❑ Os nomes das variáveis devem possuir significados
- ❑ Nomes de variáveis não podem iniciar com um número
- ❑ Após cada comando em C, deve-se colocar um ponto-e-vírgula ;
- ❑ Geralmente utiliza-se letras maiúsculas para diferenciar nomes, por exemplo:
 - ❑ pesoDoCarro
 - ❑ nomePessoa

Variáveis



- ☐ As variáveis ficam em uma posição na memória e podem ser identificadas/acessadas pelo nome definido
- ☐ Podem ter seu conteúdo alterado no decorrer do programa
- ☐ Em C, todas as variáveis possuem um tipo
- ☐ Cada tipo define os valores que a variável pode armazenar
- ☐ Cada tipo ocupa uma certa quantidade de memória

Tipos de variáveis mais utilizados

```
#include <stdio.h>

int main() {

    int numeroInteiro;

    float numeroReal;

    double numeroRealMaior;

    char letra;

    char palavra[10];

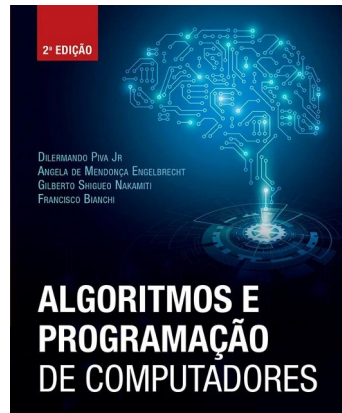
    char palavra2[5] = "teste";

    return 0;
}
```

- ✓ Variáveis do tipo **int** são variáveis que armazenam números inteiros, exemplo: 1, 10, 2050
- ✓ Variáveis do tipo **float** e **double** são variáveis que armazenam números reais, exemplo: 1.11, 20.42, 6785.903
- ✓ Variáveis do tipo **char** armazenam caracteres, exemplo: a, b, c, d
- ✓ Para armazenar uma string, a variável do tipo **char** deve ser tratada como um vetor de caracteres

Limites de tamanho

Tipo	Tamanho em bytes	Valores	Formato de i/o
char	1	-127 a 127	%c
int	2	-32768 a 32767	%d ou %i
float	4	$3.4 * 10^{-38}$ a $3.4 * 10^{38}$	%f
double	8	$1.7 * 10^{-308}$ a $1.7 * 10^{308}$	%lf



PIVA JUNIOR, Dilermando et al. Algoritmos e programação de computadores. 2. Rio de Janeiro: GEN LTC, 2019

Operadores aritméticos em C

Operador	Descrição	Exemplo
+	soma	$A + B$
-	subtração	$A - B$
*	multiplicação	$A * B$
/	divisão	A / B
%	módulo	$A \% B$
++	incremento	$A++$
--	decremento	$B--$

Conversão de portugol para C

INÍCIO ALGORITMO

Variáveis

num1, num2, soma: inteiro

escreva("Digite o valor do primeiro número:");

leia(num1);

escreva("Digite o valor do segundo número:");

leia(num2);

soma = num1 + num2;

escreva("A soma é:");

escreva(soma)

FIM ALGORITMO

```
#include <stdio.h>
```

```
int main() {
```

```
int num1, num2, soma;
```

```
printf("Digite o valor do primeiro numero: ");  
scanf("%d", &num1);
```

```
printf("Digite o valor do segundo numero: ");  
scanf("%d", &num2);
```

```
soma = num1 + num2;
```

```
printf("A soma eh: %d", soma);
```

```
return 0;
```

```
}
```

Formas de utilizar o printf

```
printf("Digite o valor do primeiro numero: ");
```

```
printf("A soma eh: %d", soma);
```

```
printf("A soma dos numeros %d e %d eh: %d", num1, num2, soma);
```

```
printf("A soma dos numeros" " %d e %d eh: %d", num1, num2, soma);
```

```
printf("%d", num2);
```

```
printf("A soma eh: %d", num1+num2);
```


Formas de utilizar o scanf

```
scanf ("%d", &num2);
```

```
scanf ("%d %d", &num1, &num2);
```

Estrutura básica de um arquivo .c

```
#include <stdio.h>
```

```
int main () {
```

```
|
```

```
return 0;
```

```
}
```

Exemplo de retorno com erro

```
#include <stdio.h>

int main() {

    int a;

    scanf("%d", a);
    printf("%d", a);

    return 0;

}
```

Exemplo de retorno com erro

10

Process returned -1073741819 (0xC0000005) execution time : 1.091 s
Press any key to continue.

Exemplo de retorno com erro

```
10
```

```
Process returned -1073741819 (0xC0000005)    execution time : 1.091 s  
Press any key to continue.
```

```
10
```

```
10
```

```
Process returned 0 (0x0)    execution time : 1.491 s  
Press any key to continue.
```